

LAPORAN PRAKTIKUM
ALGORITMA DAN PEMROGRAMAN 1
MODUL 14
SORTING



DISUSUN OLEH:
Rayhan Ahza Widyamukti
109082500210
S1 IF-13-02
DOSEN:
Wahyu Andi Saputra, S.Pd., M.Eng

PROGRAM STUDI S1 TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2024/2025

DASAR TEORI

Sorting atau pengurutan adalah proses menyusun sekumpulan data berdasarkan urutan tertentu, baik secara menaik (ascending) maupun menurun (descending). Pengurutan data merupakan salah satu operasi yang sangat penting dalam pemrograman karena dapat mempermudah pencarian, pengelompokan, dan pengolahan data. Dengan data yang telah terurut, berbagai algoritma lain seperti pencarian biner (binary search) dapat dijalankan dengan lebih efisien.

Algoritma sorting memiliki berbagai metode yang dapat digunakan sesuai dengan kebutuhan dan karakteristik data. Setiap metode memiliki cara kerja, tingkat efisiensi, serta kebutuhan memori yang berbeda. Beberapa algoritma sorting yang umum digunakan antara lain Selection Sort, Insertion Sort, Bubble Sort, Merge Sort, dan Quick Sort. Pada praktikum ini digunakan metode Selection Sort dan Insertion Sort karena algoritma tersebut mudah dipahami dan cocok untuk mempelajari konsep dasar pengurutan data.

Selection Sort adalah metode pengurutan yang bekerja dengan mencari elemen terkecil atau terbesar dari bagian data yang belum terurut, kemudian menukarkannya dengan elemen pada posisi yang seharusnya. Proses ini dilakukan berulang hingga seluruh data terurut. Algoritma ini memiliki kompleksitas waktu $O(n^2)$, sehingga kurang efisien untuk jumlah data yang sangat besar, namun cukup sederhana untuk diimplementasikan.

Insertion Sort adalah metode pengurutan yang bekerja dengan membangun bagian data yang sudah terurut secara bertahap. Setiap elemen akan dibandingkan dengan elemen-elemen sebelumnya, kemudian disisipkan pada posisi yang tepat sehingga urutan data tetap terjaga. Algoritma ini juga memiliki kompleksitas waktu $O(n^2)$ pada kasus terburuk, tetapi dapat bekerja lebih cepat pada data yang sudah hampir terurut.

Dalam implementasinya, algoritma sorting sering digunakan untuk mengurutkan data numerik, data teks, maupun data yang tersimpan dalam struktur data seperti array dan record. Penguasaan konsep sorting menjadi dasar penting dalam pengembangan perangkat lunak karena banyak permasalahan komputasi yang membutuhkan data dalam keadaan terurut agar dapat diproses secara lebih efektif dan efisien.

Soal Latihan Selection Sort

1. Rumah Kerabat

Source Code:

```
package main

import "fmt"

func selectionSort(arr []int) {
    var n int
    var i, j, min, temp int

    n = len(arr)

    for i = 0; i < n-1; i++ {
        min = i

        for j = i + 1; j < n; j++ {
            if arr[j] < arr[min] {
                min = j
            }
        }

        temp = arr[i]
        arr[i] = arr[min]
        arr[min] = temp
    }
}

func main() {
    var n int
    var m int
    var i, j int

    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Scan(&m)

        var rumah []int
        rumah = make([]int, m)

        for j = 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(rumah)

        for j = 0; j < m; j++ {
            if j > 0 {
```

```

        fmt.Print(" ")
    }
    fmt.Print(rumah[j])
}
fmt.Println()
}
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 14> go run rumahKerabat.go
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9

```

Deskripsi Program:

Program ini digunakan untuk mengurutkan nomor rumah kerabat Hercules pada setiap daerah menggunakan algoritma Selection Sort. Pertama, program membaca jumlah daerah yang akan diproses. Untuk setiap daerah, program membaca banyaknya rumah kerabat beserta nomor rumahnya dan menyimpannya ke dalam sebuah array. Selanjutnya dilakukan proses pengurutan dengan cara mencari nilai terkecil dari bagian array yang belum terurut, kemudian menukarnya dengan elemen pada posisi saat ini. Proses tersebut dilakukan berulang hingga seluruh data tersusun secara menaik. Setelah proses pengurutan selesai, nomor rumah yang sudah terurut ditampilkan dalam satu baris untuk setiap daerah sesuai dengan format keluaran yang diminta pada soal. Dengan menggunakan Selection Sort, pengurutan dapat dilakukan secara sederhana dan sistematis hingga seluruh nomor rumah berada dalam urutan dari yang terkecil ke yang terbesar.

2. Kerabat Dekat

Source Code:

```
package main

import "fmt"

func selectionSortAsc(arr []int) {
    var n int
    var i, j, min, temp int

    n = len(arr)

    for i = 0; i < n-1; i++ {
        min = i

        for j = i + 1; j < n; j++ {
            if arr[j] < arr[min] {
                min = j
            }
        }

        temp = arr[i]
        arr[i] = arr[min]
        arr[min] = temp
    }
}

func main() {
    var n int
    var m int
    var i, j int

    fmt.Scan(&n)

    for i = 0; i < n; i++ {
        fmt.Scan(&m)

        var data []int
        data = make([]int, m)

        for j = 0; j < m; j++ {
            fmt.Scan(&data[j])
        }

        selectionSortAsc(data)

        var pertama bool
        pertama = true

        for j = 0; j < m; j++ {
```

```

        if data[j]%2 != 0 {
            if !pertama {
                fmt.Print(" ")
            }
            fmt.Print(data[j])
            pertama = false
        }
    }

    for j = m - 1; j >= 0; j-- {
        if data[j]%2 == 0 {
            if !pertama {
                fmt.Print(" ")
            }
            fmt.Print(data[j])
            pertama = false
        }
    }

    fmt.Println()
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 14> go run kerabatDekat.go
3
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4

```

Deskripsi Program:

Program ini digunakan untuk menampilkan nomor rumah kerabat Hercules dengan aturan khusus agar jumlah penyeberangan jalan dapat diminimalkan. Nomor rumah ganjil dianggap berada di sisi kiri jalan, sedangkan nomor rumah genap berada di sisi kanan jalan. Program terlebih dahulu membaca jumlah daerah dan nomor rumah pada setiap daerah, kemudian mengurutkan seluruh nomor rumah secara menaik menggunakan algoritma Selection Sort. Setelah data terurut, program menampilkan semua nomor rumah ganjil terlebih dahulu dalam urutan menaik. Selanjutnya, program menampilkan semua nomor rumah genap dalam urutan menurun dengan menelusuri array dari belakang ke depan. Dengan cara ini, keluaran yang dihasilkan sesuai dengan

ketentuan soal, yaitu nomor ganjil terurut membesar diikuti nomor genap terurut mengecil untuk setiap daerah yang diproses.

3. Median

Source Code:

```
package main

import "fmt"

func insertionSort(arr []int) {
    var i, j, key int

    for i = 1; i < len(arr); i++ {
        key = arr[i]
        j = i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func median(arr []int) int {
    var n int
    n = len(arr)

    if n%2 == 1 {
        return arr[n/2]
    }

    return (arr[n/2-1] + arr[n/2]) / 2
}

func main() {
    var data []int
    var angka int

    data = make([]int, 0)

    for {
        fmt.Scan(&angka)

        if angka == -5313 {
            break
        }
    }
}
```

```

    }

    if angka == 0 {
        var salinan []int
        salinan = make([]int, len(data))
        copy(salinan, data)

        insertionSort(salinan)
        fmt.Println(median(salinan))
    } else {
        data = append(data, angka)
    }
}
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 14> go run median.go
7 23 11 0 5 19 2 29 3 13 17 0 -5313
11
12

```

Deskripsi Program:

Program ini membaca serangkaian bilangan bulat hingga menemukan nilai penanda akhir yaitu -5313. Setiap bilangan yang bukan 0 dan bukan -5313 akan disimpan ke dalam array. Ketika program menemukan angka 0, seluruh data yang telah tersimpan sampai saat itu akan disalin dan diurutkan menggunakan algoritma Insertion Sort sesuai petunjuk pada soal. Setelah data terurut, program menghitung nilai median. Jika jumlah data ganjil, median adalah nilai yang berada tepat di tengah data yang telah terurut. Jika jumlah data genap, median diperoleh dari rata-rata dua nilai tengah dan hasil pembagian dibulatkan ke bawah karena menggunakan tipe data integer. Nilai median kemudian dicetak ke layar setiap kali angka 0 ditemukan. Proses ini berlanjut hingga program membaca angka -5313 yang menandakan akhir masukan.

Soal Latihan Insertion Sort

1. Baca Data

Source Code:

```
package main

import "fmt"

func insertionSort(arr []int) {
    var i, j, key int

    for i = 1; i < len(arr); i++ {
        key = arr[i]
        j = i - 1

        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }

        arr[j+1] = key
    }
}

func main() {
    var data []int
    var bilangan int

    data = make([]int, 0)

    for {
        fmt.Scan(&bilangan)

        if bilangan < 0 {
            break
        }

        data = append(data, bilangan)
    }

    insertionSort(data)

    var i int

    for i = 0; i < len(data); i++ {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(data[i])
    }
}
```

```

    fmt.Println()

    if len(data) <= 2 {
        fmt.Println("Data berjarak tetap")
        return
    }

    var jarak int
    var tetap bool

    jarak = data[1] - data[0]
    tetap = true

    for i = 2; i < len(data); i++ {
        if data[i]-data[i-1] != jarak {
            tetap = false
            break
        }
    }

    if tetap {
        fmt.Println("Data berjarak", jarak)
    } else {
        fmt.Println("Data berjarak tidak tetap")
    }
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 14> go run bacaData.go
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6

```

Deskripsi Program:

Program ini membaca sekumpulan bilangan bulat hingga ditemukan bilangan negatif sebagai penanda akhir masukan. Hanya bilangan non-negatif yang disimpan ke dalam array. Setelah seluruh data terbaca, array diurutkan menggunakan algoritma Insertion Sort dengan cara menyisipkan setiap elemen ke posisi yang sesuai pada bagian array yang sudah terurut. Setelah pengurutan selesai, isi array yang telah terurut ditampilkan pada baris pertama. Selanjutnya program memeriksa apakah selisih antara setiap dua elemen yang berurutan selalu sama. Jika seluruh selisih bernilai sama, maka data dianggap memiliki jarak tetap dan program menampilkan pesan "Data berjarak x", dengan x adalah nilai selisih tersebut. Jika terdapat perbedaan selisih pada salah satu pasangan elemen, maka program menampilkan pesan "Data berjarak tidak tetap". Pada contoh pertama jaraknya selalu 6 sehingga menghasilkan keluaran "Data berjarak 6",

sedangkan pada contoh kedua selisih antar elemen tidak sama sehingga menghasilkan keluaran "Data berjarak tidak tetap".

2. Perpustakaan

Source Code:

```
package main

import "fmt"

type Buku struct {
    id, judul, penulis, penerbit string
    eksemplar, tahun, rating      int
}

func DaftarkanBuku(pustaka []Buku, n int) {
    var i int

    for i = 0; i < n; i++ {
        fmt.Scan(
            &pustaka[i].id,
            &pustaka[i].judul,
            &pustaka[i].penulis,
            &pustaka[i].penerbit,
            &pustaka[i].eksemplar,
            &pustaka[i].tahun,
            &pustaka[i].rating,
        )
    }
}

func CetakTerfavorit(pustaka []Buku, n int) {
    var i, idx int

    idx = 0

    for i = 1; i < n; i++ {
        if pustaka[i].rating > pustaka[idx].rating {
            idx = i
        }
    }

    fmt.Println(
        pustaka[idx].judul,
        pustaka[idx].penulis,
        pustaka[idx].penerbit,
        pustaka[idx].tahun,
    )
}
```

```

func UrutBuku(pustaka []Buku, n int) {
    var i, j int
    var key Buku

    for i = 1; i < n; i++ {
        key = pustaka[i]
        j = i - 1

        for j >= 0 && pustaka[j].rating < key.rating {
            pustaka[j+1] = pustaka[j]
            j--
        }

        pustaka[j+1] = key
    }
}

func Cetak5Terbaru(pustaka []Buku, n int) {
    var i, batas int

    if n < 5 {
        batas = n
    } else {
        batas = 5
    }

    for i = 0; i < batas; i++ {
        fmt.Println(pustaka[i].judul)
    }
}

func CariBuku(pustaka []Buku, n, r int) {
    var kiri, kanan, tengah int

    kiri = 0
    kanan = n - 1

    for kiri <= kanan {
        tengah = (kiri + kanan) / 2

        if pustaka[tengah].rating == r {
            fmt.Println(
                pustaka[tengah].judul,
                pustaka[tengah].penulis,
                pustaka[tengah].penerbit,
                pustaka[tengah].tahun,
                pustaka[tengah].eksemplar,
                pustaka[tengah].rating,
            )
            return
        }
    }
}

```

```

        }

        if pustaka[tengah].rating < r {
            kanan = tengah - 1
        } else {
            kiri = tengah + 1
        }
    }

    fmt.Println("Tidak ada buku dengan rating seperti
itu")
}

func main() {
    var n, ratingCari int

    fmt.Scan(&n)

    var pustaka []Buku
    pustaka = make([]Buku, n)

    DaftarkanBuku(pustaka, n)

    fmt.Scan(&ratingCari)

    CetakTerfavorit(pustaka, n)

    UrutBuku(pustaka, n)

    Cetak5Terbaru(pustaka, n)

    CariBuku(pustaka, n, ratingCari)
}

```

Output:

```

PS D:\Materi Kuliah\Semester 2\Alpro Praktikum\Modul\Modul 14> go run perpustakaan.go
1
B01 LaskarPelangi AndreaBentata Bentang 20 2005 95
95
LaskarPelangi AndreaBentata Bentang 2005
LaskarPelangi
LaskarPelangi AndreaBentata Bentang 2005 20 95

```

Deskripsi Program:

Program ini digunakan untuk mengelola data buku dalam sebuah perpustakaan. Mula-mula program membaca sejumlah data buku yang terdiri dari id, judul, penulis, penerbit, jumlah eksemplar, tahun terbit, dan rating. Setelah seluruh data tersimpan, program mencari buku favorit dengan membandingkan rating seluruh buku dan menampilkan buku yang memiliki rating tertinggi. Selanjutnya data buku diurutkan secara menurun berdasarkan rating menggunakan algoritma Insertion Sort sesuai ketentuan soal. Setelah terurut, program menampilkan maksimal lima judul buku dengan rating tertinggi. Terakhir, program melakukan pencarian buku berdasarkan rating yang dimasukkan pengguna menggunakan metode Binary Search karena data sudah dalam keadaan terurut. Jika ditemukan, seluruh informasi buku tersebut ditampilkan, sedangkan jika tidak ditemukan program akan menampilkan pesan bahwa tidak ada buku dengan rating yang dimaksud.

DAFTAR PUSTAKA

Modul Praktikum Algoritma dan Pemrograman 2 – Modul 14 Sorting. Fakultas Informatika, Telkom University Purwokerto.

Noval Agung. Dasar Pemrograman Golang.

<https://dasarpemrogramangolang.novalagung.com>

Kadir, Abdul. 2018. Dasar Algoritma dan Pemrograman Menggunakan Go. Yogyakarta: Andi Offset.